Алматинский Технологический Университет (АТУ)

Образовательная программа «Информационные системы», 3 курс

# ОТЧЁТ ПО САМОСТОЯТЕЛЬНОЙ РАБОТЕ СТУДЕНТА

по дисциплине «Проектирование и применение NoSQL-технологий»

Мини-проект, вариант 12:

**«Телеметрия умного дома» (Apache Cassandra)**

Выполнил: Оразбаев Махсуд, группа ИС 24-22

Проверил: Рашидинов Д.Р.

2026

# 1. Цель и постановка задачи

Цель — спроектировать и реализовать прототип подсистемы хранения телеметрии умного дома на Apache Cassandra: таблицы по устройству и временному bucket, выборка измерений по периоду, контроль размера партиций.

Объект — потоки измерений датчиков (температура, влажность, $CO_2$, потребляемая мощность). Нужно принимать измерения, хранить историю 30 дней, выдавать текущее состояние, данные за период и последние N значений, считать агрегаты и фиксировать выход значений за пороги. Реализация выполнена на Python с драйвером `cassandra-driver`; стенд поднимается через Docker Compose.

Задачи: сформулировать требования и Access Patterns; обосновать выбор СУБД (ADR); спроектировать модель данных; реализовать код, генерацию синтетических данных, 10 демонстрационных запросов и тесты; оформить репозиторий, отчёт и презентацию.

# 2. Требования

### Функциональные требования

| ID | Требование |
|---|---|
| FR1 | Регистрация устройства: комната, тип, метрика, единица измерения, пороги min/max. |
| FR2 | Получение списка устройств комнаты. |
| FR3 | Приём измерения (device_id, ts, value) и сохранение в историю. |
| FR4 | Выдача текущего (последнего) состояния устройства. |
| FR5 | Выборка измерений устройства за произвольный период, в том числе через границу суток. |
| FR6 | Выборка последних N измерений устройства. |
| FR7 | Агрегаты за период: количество, среднее, минимум, максимум. |
| FR8 | Автоматическая тревога при выходе значения за пороги; просмотр тревог дома за день. |
| FR9 | Удаление устройства вместе с его историей. |

### Нефункциональные требования

| ID | Требование |
|---|---|
| NFR1 | Масштабируемость записи: запись идёт в одну партицию по ключу, без межузловых операций. |
| NFR2 | Ограничение размера партиции (ориентир: до 100 тыс. строк и до 100 МБ); bucket настраивается (день/час). |
| NFR3 | Автоматическое истечение данных через 30 дней (TTL) без ручной очистки. |
| NFR4 | Воспроизводимость: запуск стенда командой docker compose и двумя скриптами. |
| NFR5 | Отказоустойчивость: репликация данных (RF=1 на учебном стенде, RF=3 в промышленной схеме). |
| NFR6 | Низкая задержка чтения свежих данных: «последние N» и «текущее состояние» читают одну партицию/строку. |

# 3. Предметная область и Access Patterns

Предметная область: дом содержит комнаты; в комнатах установлены устройства, каждое измеряет одну метрику с заданными порогами. Устройство периодически (в учебной выборке — раз в 15 минут) отправляет пару «время — значение». Данные неизменяемы; чтение почти всегда ограничено одним устройством и интервалом времени. Такой профиль соответствует модели «широкая строка во временной партиции».

| ID | Запрос | Тип | Приоритет | Таблица | Ожидаемый результат |
|---|---|---|---|---|---|
| AP1 | Регистрация устройства | запись | High | `devices_by_room`<br>`devices_by_id` | Устройство видно в списке комнаты и находится по id. |
| AP2 | Устройства комнаты | чтение | Medium | `devices_by_room` | Список устройств по ключу (home_id, room), отсортирован по device_id. |
| AP3 | Запись измерения | запись | Critical | `measurements_by_device_bucket`<br>`latest_by_device`<br>`alerts_by_home_day` | Строка в партиции (device_id, bucket); обновлено текущее значение; при выходе за порог создана тревога. |
| AP4 | Измерения за период | чтение | High | `measurements_by_device_bucket` | Строки периода, новые первыми; запрос по одной партиции на каждый bucket периода. |
| AP5 | Последние N измерений | чтение | High | `measurements_by_device_bucket` | N последних строк (LIMIT N) из актуальной партиции, при нехватке — из предыдущих. |
| AP6 | Текущее состояние | чтение | Critical | `latest_by_device` | Одна строка по первичному ключу device_id. |
| AP7 | Агрегаты за период | чтение | Medium | `measurements_by_device_bucket` | count/avg/min/max: CQL-агрегаты по каждой партиции и объединение на стороне приложения. |
| AP8 | Тревоги дома за день | чтение | High | `alerts_by_home_day` | Список тревог дня, новые первыми. |
| AP9 | Удаление устройства | удаление | Low | `все таблицы` | Удалены партиции измерений, текущее состояние и записи справочников. |
| AP10 | Размер партиции | чтение | Low | `measurements_by_device_bucket` | COUNT(*) по (device_id, bucket) для контроля large partition. |

## 4. Сравнение альтернатив и ADR

| Вариант | Плюсы | Минусы для задачи |
|---|---|---|
| PostgreSQL / TimescaleDB | SQL, JOIN, транзакции, простота | Запись масштабируется вертикально; шардирование сложнее |
| MongoDB time-series | Агрегации, гибкая схема | Шардирование требует выбора ключа и балансировки |
| Redis TimeSeries | Очень низкая задержка | Объём ограничен RAM; месяцы истории дороги |
| InfluxDB | Специализирован под метрики | Кластеризация в OSS ограничена; иная модель запросов |
| **Cassandra** | Линейное масштабирование записи, репликация, TTL, сортировка в партиции | Нет JOIN и ad-hoc запросов; модель строится под запросы |

**ADR-001. Решение:** использовать Apache Cassandra и принцип «таблица под запрос». Профиль нагрузки — append-only запись и чтение «устройство + период» — совпадает с сильными сторонами Cassandra. Альтернативы лучше там, где нужны JOIN, транзакции или произвольная аналитика.

**Выбор partition key.** Ключ `(device_id)` отклонён: партиция растёт бесконечно. Ключ `(home_id, day)` для измерений отклонён: все устройства дома пишут в одну партицию (hot partition). Принят `(device_id, bucket)`, где bucket — сутки; при шаге 1 мин это ≈ 1 440 строк в партиции. Для частых датчиков включается почасовой bucket параметром

BUCKET_GRANULARITY=hour.

**Последствия.** Запросы вне модели (например, WHERE value > 25) требуют ALLOW FILTERING и не поддерживаются; запись в несколько таблиц неатомарна; межустройственная аналитика выносится за пределы Cassandra. Для одного дома с десятком датчиков PostgreSQL/TimescaleDB проще — Cassandra оправдана при масштабе платформы. Полный текст: docs/ADR.md.

## 5. Схема данных и архитектура

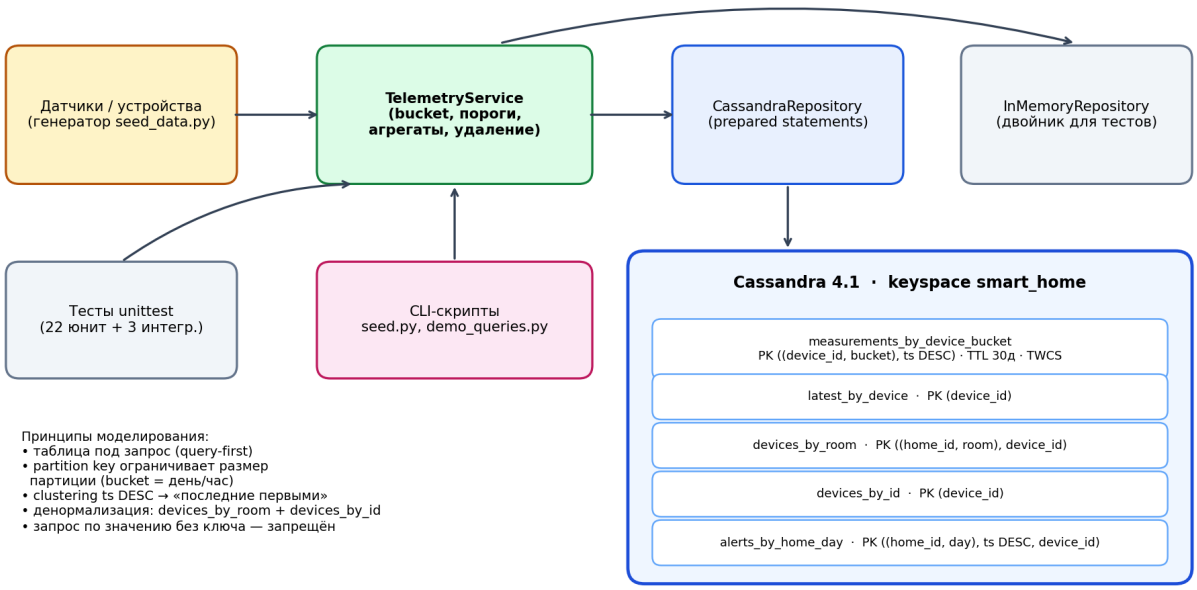



*Рис. 1. Архитектура прототипа и таблицы keyspace smart_home*

| Таблица | Primary key | Назначение |
|---|---|---|
| measurements_by_device_bucket | ((device_id, bucket), ts DESC) | История; TTL 30 дней; TWCS с окном 1 день |
| latest_by_device | device_id | Текущее состояние (upsert) |
| devices_by_room | ((home_id, room), device_id) | Устройства комнаты |
| devices_by_id | device_id | Поиск устройства и порогов |
| alerts_by_home_day | ((home_id, day), ts DESC, device_id) | Тревоги за день |

### Оценка размера партиции

| Частота измерений | Строк в суточной партиции | Оценка размера* | Решение |
|---|---|---|---|
| 1 раз в 15 мин (учебная выборка) | 96 (проверено тестом) | ≈ 5 КБ | bucket = день |
| 1 раз в минуту | 1 440 | ≈ 70 КБ | bucket = день |
| 1 раз в 10 секунд | 8 640 | ≈ 430 КБ | bucket = день |
| 1 раз в секунду | 86 400 | ≈ 4 МБ | день допустим; час (3 600 строк, ≈ 180 КБ) удобнее для чтения |

** Грубая оценка при ≈ 50 байт на строку с учётом служебных данных; фактический размер определяется nodetool tablestats на реальном стенде. Во всех режимах значения на порядки ниже ориентира в 100 МБ, поэтому суточный bucket подходит по умолчанию, а почасовой включается параметром без изменения кода*

*запросов.*

## 6. Реализация и фрагменты кода

Код разделён на слои: `service.py` — бизнес-логика, `cassandra_repo.py` — доступ к Cassandra через prepared statements, `memory_repo.py` — двойник с той же семантикой партиций и сортировки для тестов без БД. Схема хранится в `schema.cql`.

### Запись измерения: вычисляется bucket, обновляются история и текущее состояние, проверяются пороги.

```
def record(self, device_id: str, ts: datetime, value: float) -> Optional[Alert]:
    dev = self.repo.get_device(device_id)
    if dev is None:
        raise KeyError(f"unknown device: {device_id}")
    self.repo.insert_measurement(device_id, bucket_for(ts, self.gran), ts, value)
    cur = self.repo.get_latest(device_id)
    if cur is None or ts >= cur.ts:          # не затираем свежие данные опоздавшими
        self.repo.upsert_latest(device_id, ts, value)
    alert = self._check_thresholds(dev, ts, value)
    if alert:
        self.repo.insert_alert(alert)
    return alert
```

### Выборка за период: обход bucket от новых к старым; внутри партиции порядок DESC задан CLUSTERING ORDER.

```
def period(self, device_id: str, start: datetime, end: datetime,
           newest_first: bool = True) -> list[Measurement]:
    buckets = buckets_between(start, end, self.gran)
    out: list[Measurement] = []
    for b in reversed(buckets):              # каждая партиция уже DESC
        out.extend(self.repo.measurements_in_bucket(device_id, b, start, end))
    return out if newest_first else out[::-1]
```

### Основная таблица (фрагмент `schema.cql`):

```
CREATE TABLE IF NOT EXISTS measurements_by_device_bucket (
  device_id text, bucket text, ts timestamp, value double,
  PRIMARY KEY ((device_id, bucket), ts)
) WITH CLUSTERING ORDER BY (ts DESC)
  AND default_time_to_live = 2592000
  AND compaction = {'class': 'TimeWindowCompactionStrategy',
                    'compaction_window_unit': 'DAYS', 'compaction_window_size': 1};
```

### Агрегаты считаются самой Cassandra по каждой партиции и объединяются взвешенным средним:

```
SELECT COUNT(*) AS n, AVG(value) AS avg, MIN(value) AS min, MAX(value) AS max
FROM measurements_by_device_bucket
WHERE device_id=? AND bucket=? AND ts>=? AND ts<=?
```

### Соответствие Access Patterns и CQL-запросов

| AP | CQL (prepared statement) | Ключ в WHERE |
|---|---|---|
| AP2 | SELECT * FROM devices_by_room WHERE home_id=? AND room=? | вся партиция |
| AP3 | INSERT INTO measurements_by_device_bucket ...; INSERT INTO latest_by_device ...; INSERT INTO alerts_by_home_day ... (при выходе за порог) | полный PK |
| AP4 | SELECT ... FROM measurements_by_device_bucket WHERE device_id=? AND bucket=? AND ts>=? AND ts<=? (по одному запросу на bucket) | партиция + диапазон clustering |
| AP5 | SELECT ... WHERE device_id=? AND bucket=? LIMIT ? | партиция |
| AP6 | SELECT ... FROM latest_by_device WHERE device_id=? | полный PK |
| AP7 | SELECT COUNT(*), AVG(value), MIN(value), MAX(value) ... WHERE device_id=? AND bucket=? AND ts>=? AND ts<=? | партиция + диапазон |
| AP8 | SELECT ... FROM alerts_by_home_day WHERE home_id=? AND day=? | партиция |
| AP9 | DELETE FROM measurements_by_device_bucket WHERE device_id=? AND bucket=? (по bucket); DELETE из latest_by_device, devices_by_room, devices_by_id | партиция / PK |
| AP10 | SELECT COUNT(*) ... WHERE device_id=? AND bucket=? | партиция |

Данные: скрипт `scripts/seed.py` создаёт схему, 7 устройств в 3 комнатах и 2 023 синтетических измерения за 3 суток (детерминированный seed=42, суточная синусоида + шум, около 0,5 % выбросов для проверки тревог). Образец из 100 событий лежит в `data/sample_events.csv`. Реальные персональные данные не используются.

### Устройства синтетического набора

| device_id | Комната | Метрика | Норма (min...max) |
|---|---|---|---|
| `thermo-living-01` | living | temperature, °C | 16.0 ... 28.0 |
| `hum-living-01` | living | humidity, % | 25.0 ... 70.0 |
| `thermo-bedroom-01` | bedroom | temperature, °C | 16.0 ... 28.0 |
| `co2-bedroom-01` | bedroom | co2, ppm | — ... 1200.0 |
| `thermo-kitchen-01` | kitchen | temperature, °C | 16.0 ... 30.0 |
| `power-kitchen-01` | kitchen | power, W | — ... 3000.0 |
| `fridge-kitchen-01` | kitchen | temperature, °C | 0.0 ... 7.0 |

## 7. Запросы и результаты

Демонстрация состоит из 10 запросов (`scripts/demo_queries.py`). Нетривиальные: Q5 — период через полночь (две партиции, сохранение порядка DESC); Q6 — агрегаты за 24 часа по нескольким партициям; Q8 — средняя температура по дням и комнатам (связь справочника и измерений выполняется в приложении, так как JOIN в Cassandra отсутствует). Q10 демонстрирует создание, чтение, изменение (upsert по тому же ключу) и удаление.

```
=== Q1. Устройства в комнате kitchen (AP2)
  fridge-kitchen-01    temperature  [0.0..7.0] °C
  power-kitchen-01     power        [None..3000.0] W
  thermo-kitchen-01    temperature  [16.0..30.0] °C
=== Q2. Текущее состояние холодильника (AP6)
  2026-10-18 12:45 -> 4.81
=== Q3. Последние 5 измерений термостата гостиной (AP5)
  2026-10-18 12:45  23.68
  2026-10-18 12:30  23.41
  2026-10-18 12:15  23.7
  2026-10-18 12:00  23.5
  2026-10-18 11:45  23.03
=== Q4. Измерения за 2 часа в пределах одной партиции (AP4)
  строк: 9, первая: 02:00, последняя: 00:00
=== Q5. НЕТРИВИАЛЬНЫЙ: период через полночь (2 партиции, порядок DESC) (AP4)
  партиции: ['2026-10-17', '2026-10-18'], строк: 9
  18 01:00, 18 00:45, 18 00:30, 18 00:15, 18 00:00, 17 23:45, 17 23:30, 17 23:15, 17 23:00
```

```
=== Q6. НЕТРИВИАЛЬНЫЙ: агрегаты за 24 часа (CQL COUNT/AVG/MIN/MAX по партициям) (AP7)
  count=97 avg=22.02 min=19.73 max=24.3
=== Q7. Тревоги за сегодня и вчера (AP8)
  2026-10-18 01:30 hum-living-01        humidity 78.0% выше порога 70.0%
  2026-10-17 18:00 co2-bedroom-01       co2 1450.0ppm выше порога 1200.0ppm
  2026-10-17 13:45 power-kitchen-01     power 3400.0W выше порога 3000.0W
  2026-10-17 10:30 thermo-living-01     temperature 30.0°C выше порога 28.0°C
  2026-10-17 10:00 thermo-kitchen-01    temperature 32.5°C выше порога 30.0°C
=== Q8. НЕТРИВИАЛЬНЫЙ: средняя температура по дням и комнатам (join на стороне приложения)
  living   thermo-living-01     10-16: 22.0 | 10-17: 22.1 | 10-18: 21.3
  bedroom  thermo-bedroom-01    10-16: 20.0 | 10-17: 20.0 | 10-18: 19.2
  kitchen  fridge-kitchen-01    10-16: 4.0 | 10-17: 4.0 | 10-18: 3.6
  kitchen  thermo-kitchen-01    10-16: 23.0 | 10-17: 23.1 | 10-18: 22.0
=== Q9. Размер партиций (контроль large partition)
  (thermo-living-01, 2026-10-16): 96 строк
  (thermo-living-01, 2026-10-17): 96 строк
  (thermo-living-01, 2026-10-18): 52 строк
=== Q10. CRUD-цикл: создание → чтение → изменение → удаление
  C: записано 35.0, тревога: temperature 35.0°C выше порога 30.0°C
  R: current = 35.0
  U: после перезаписи значение = 21.5
  D: удалено партиций: 3, current = None
```

*Вывод получен на Apache Cassandra 4.1 (Docker, один узел, RF=1) после* `seed.py --reset`*; Python 3.14, cassandra-driver 3.30.1. Данные генерируются относительно времени запуска, поэтому при повторе значения будут немного отличаться.*

Ограничение модели показывают отклонённые запросы: `SELECT … WHERE value > 25` и поиск измерений по комнате без указания устройства не принимаются Cassandra без ALLOW FILTERING — для них нужны отдельные таблицы под запрос.

# 8. Тестирование

Всего 25 тестов (`unittest`): 22 юнит-теста бизнес-логики на in-memory репозитории, 3 интеграционных теста на живой Cassandra (отдельный keyspace `smart_home_test`).

| № | Сценарий | Проверка |
|---|---|---|
| 1 | Период через полночь | 9 строк из двух партиций, порядок DESC; режим ASC |
| 2 | Агрегаты по партициям | count/avg/min/max совпадают с ручным расчётом; пустой период даёт count=0 |
| 3 | Последние N | 60 записей собираются из 2 партиций, самая новая — первая |
| 4 | Тревога по порогу | значение выше max / ниже min создаёт запись в alerts; норма — не создаёт |
| 5 | Запоздавшее событие | current не затирается более старой записью |
| 6 | Upsert по PK | повторная запись с тем же ts заменяет значение |
| 7 | Удаление устройства | очищены измерения, current и справочники |
| 8 | Часовой bucket | 4 записи в партиции; период и last_n корректны |
| + | Граничные случаи | неизвестное устройство, неверный диапазон, неизвестная гранулярность |

Результат полного запуска на Cassandra 4.1: `Ran 25 tests … OK`, включая 3 интеграционных теста (`RUN_INTEGRATION=1`; около 93 с на одноузловом стенде в Docker). Без этой переменной интеграционные тесты пропускаются, остальные 22 выполняются за доли секунды. Интеграционные тесты проверяют ту же логику на реальных CQL-запросах: сортировку партиции, CQL-агрегаты и запись тревог.

### Запуск и воспроизводимость

```
cp .env.example .env
docker compose up -d
pip install -r requirements.txt
python scripts/seed.py --reset
python scripts/demo_queries.py
python -m unittest discover -s tests -t . -v
RUN_INTEGRATION=1 python -m unittest tests.test_integration -v
```

В репозитории нет секретов: `.env.example` содержит только адрес, порт, имя keyspace и гранулярность bucket; `.env` исключён через `.gitignore`.

### Консистентность и эксплуатация

**Уровни согласованности.** Для телеметрии допустима запись с уровнем `ONE`/`LOCAL_ONE`: потеря единичного измерения при сбое узла приемлема, а запись остаётся быстрой. Чтение текущего состояния и тревог в промышленной схеме (RF=3) разумно выполнять с `LOCAL_QUORUM`. В прототипе (один узел) используется уровень драйвера по умолчанию, поэтому поведение при отказах и расхождении реплик здесь не проверялось.

**Идемпотентность.** Все вставки — upsert по первичному ключу: повторная отправка того же измерения не создаёт дубликата. Это позволяет безопасно повторять запись после таймаута, а запоздавшие события не затирают `latest_by_device`.

**Мониторинг.** Размер и рост партиций проверяются командами `nodetool tablestats` и `nodetool tablehistograms`; в прототипе для этого есть функция `partition_size()` (запрос Q9). Признаки проблем: партиции больше ориентира, рост числа tombstone, задержки чтения в одной партиции. Срок хранения (30 дней) и размер окна компактизации согласованы: окно в 1 день создаёт примерно 30 групп SSTable, истекающих целиком.

## 9. Ограничения

- Запись в несколько таблиц неатомарна; сбой между вставками возможен. Операции идемпотентны, повтор безопасен; для строгой согласованности нужен LOGGED BATCH или фоновая сверка.
- `record()` читает справочник и текущее состояние перед записью (чтение по первичному ключу, но не «чистая» запись).
- Удаления создают tombstone; массовое удаление устройства за длинный период нужно планировать.
- `COUNT(*)` по партиции читает все её строки — для мониторинга размера подходит, для горячего пути нет.
- Аналитика между устройствами и произвольные фильтры не поддерживаются моделью; для них требуется отдельный аналитический контур.
- Стенд одноузловой, RF=1: отказоустойчивость и tunable consistency не проверялись; данные синтетические.

## 10. Выводы

- Модель «таблица под запрос» с partition key (device_id, bucket) закрывает 10 Access Patterns без ALLOW FILTERING; размер партиции контролируется настройкой bucket.
- Clustering order ts DESC делает чтение «последних N» и «текущего состояния» обращением к одной партиции или строке.
- TTL и TimeWindowCompactionStrategy снимают задачу очистки: истёкшие данные уходят целыми SSTable.
- Цена решения — денормализация и отсутствие ad-hoc запросов; для малых объёмов реляционная СУБД проще, Cassandra оправдана при росте нагрузки и числа устройств.
- Разделение на сервис и репозиторий позволило проверить логику без БД и оставить единые демонстрационные запросы для обоих режимов.

## 11. Ссылка на GitHub

[вставьте ссылку на репозиторий] — README, исходники, тестовые данные, ADR, отчёт и презентация.